Hashira Search — Final Project Report

Aman Sa, Ashay Katre, Benjamin Pusch, Vinay Padegal
CIS 5550: Internet and Web Systems

Abstract

Hashira Search is an end-to-end distributed web search engine built on a custom stack: a sharded persistent Key–Value Store (KVS), a Spark-like computation layer (Flame), and a search pipeline including crawling, metadata extraction, indexing, PageRank, and an interactive UI with multiple verticals (web, images, PDFs, videos). Our focus was speed first for scalability via streaming, batching, and parallel execution and then accuracy and performance.

1 Summary Items

1) Extra Credit Features Implemented

We implemented the following extra-credit features (beyond the baseline requirements):

- **Image search** (first-class vertical) using anchor/alt/nearby text context.
- **PDF search** (vertical) using link/anchor context (with approximate positions for proximity).
- **Video search** (vertical) using anchor/caption/ARIA/nearby context.
- **Infinite scrolling / lazy loading** with server-side incremental scoring and caching.
- **Spell correction** ("Did you mean ..." + "Search instead for ...") integrated into query flow.
- **Search suggestions** (type-ahead completions from indexed phrases/terms).
- **Weather feature** (task-specific result flow) with a dedicated `/weather` view.
- **Debug mode** that surfaces scoring components (TF/IDF, PageRank, etc.) for diagnostics.
- **Extremely Efficient reads/writes** We try to maximize performance with this batched and pool architecture. Diagram: 2.

2) Changes Made After Code-Complete Deadline

After code-complete, we made changes primarily to ensure a stable and performant demo at larger scale:

- **Performance + scalability upgrades**: deep batching/bulk operations in KVS + Flame, streaming scans, and end-to-end parallelization of crawl/metadata/index/PageRank jobs.
- **Corpus scaling changes**: initial ∼10k-page validation, later larger crawls; after encountering crawl "rabbit holes" at very large scale, we re-crawled a clean ∼100k corpus using blacklisting, stricter frontier filtering.
- **Feature additions / polish**: PDF and video verticals, tabbed UI, infinite scroll flow, weather flow
- **Robustness fixes**: edge-case handling in bulk endpoints/ batched write paths; stability improvements

2 Introduction

Approach. Built as a layered distributed system: (1) store everything in sharded KVS tables; (2) run all expensive transformations in Flame using table-backed RDDs; and (3) keep online query serving responsive through careful ranking design and incremental scoring.

Division of labor. Aman and Ben: Crawler + KVS + robustness, storage throughput, Flame parallel execution and batch I/O. **Vinay and Ashay**: Indexing + ranking, PageRank, SearchServer, UI, EC integrations, metadata.

3 Architecture

3.1 System Overview

Our architecture consists of three layers:

- **Crawler + MetaDataExtractor** downloads and parses pages
- **Indexers** produce inverted indexes for web and media verticals
- **SearchServer + UI** executes online query flow with ranking and caching

All components interact through a **KVS Layer** (sharded tables, bulk ops, streaming scans) and **Flame Layer** (table-backed RDDs, parallel lambdas). See Appendix B for architecture diagram and detailed component interactions.

3.2 Data Storage

All data is stored as `Row` objects with persistent tables (`pt-` prefix) for restartability. Core tables include:

- `pt-crawl` (HTML)
- `pt-meta-info` (titles/descriptions)
- `pt-outlinks` (link graph)
- `pt-anchors` (inbound anchor text)
- `pt-index` (web inverted index)
- `pt-docmagnitudes` (cosine normalization)
- media link tables (`pt-image-links`, `pt-pdf-links`, `pt-video-links`)
- media indexes
- `pt-pageranks`

Complete table descriptions in Appendix B.3.

3.3 Design Rationale

KVS-backed tables: Every intermediate result as a KVS table makes Flame operations stateless and restartable; large datasets processed via streaming without fitting in memory.
Streaming + batching: Prevents OOM and dramatically reduces HTTP/disk overhead.
Two-level parallelism:

- Worker-level: partitions across Flame workers
- Thread-level: bounded-queue executor per worker for CPU-heavy lambdas with shared batched writer

Scalability: Horizontal sharding, parallel jobs, bulk endpoints (`bulkPutRows`, `bulkGetRows`).
Fault tolerance: Intermediate results in tables enable restartable stages; known limitation is lack of automatic replication.

4 Ranking

4.1 Ranking Features and Scoring

Web results combine:

- Cosine TF–IDF similarity
- Field-based signals (title/meta/headers/body)
- Proximity score (tight matches from stored positions)
- URL + inbound anchor signal
- PageRank

Media verticals rank by anchor/caption context relevance without PageRank.

Final web ranking:

$$\text{finalScore} = 0.45 \cdot \text{cosine} + 0.15 \cdot \text{field} + 0.10 \cdot \text{proximity} + 0.10 \cdot \text{urlAnchor} + 0.20 \cdot \text{pageRankNorm}$$

Media formulas in Appendix E.

4.2 Query Flow

Spell correction: Per-token correction with "Did you mean ..." and "Search instead for ..." unless user forces original.
Search suggestions: Type-ahead completions from indexed tokens/phrases.
Lazy scoring + caching: Score top slice synchronously, continue in background, cache by corrected query for infinite scroll reuse.
Detailed scoring formulas and optimization strategies in Appendix E.

5 Evaluation

5.1 Methodology

All experiments ran on EC2 deployment with separate instances for KVS/Flame workers and frontend. Reproducibility requires: worker counts, threads per worker, seed URLs + blacklist config, and exact input/output tables per stage. We report throughput as **items/sec** and **wall-clock time**.

5.2 Scale and Performance

Validated on ∼10k-page corpus, then attempted larger crawl of 500k items; encountered rabbit-hole behavior at very large scale but completed the implementation. Re-crawled clean ∼100k corpus using stricter filtering and blacklisting, completing in **under one hour** (>28 pages/sec end-to-end average). Largest wins: thread-level parallel lambdas (4–8× speedup on CPU-bound tasks), BatchedKVSClient (reduced HTTP volume), KVS BulkWritePool (async batching), and streaming scans (stable memory footprint). Detailed performance metrics, PageRank experiments, and query latency analysis in Appendix F.

6 Lessons Learned

Success: Delivered fully working distributed search engine with multi-vertical search, scalable batch jobs, and interactive UI.
What worked:

- KVS-backed tables for scalability and restartability
- Batching + streaming + parallelism for **BIGGEST PERFORMANCE** gains !!(go through) - 2.
- Lazy scoring + caching for fast UI

Challenges:

- Large-scale crawling requires strict filtering to avoid rabbit holes
- Concurrency + bulk writes need careful handling (merging semantics, request sizing, backpressure)
- Lack of replication limits true fault tolerance

Future improvements:

- Add replication/checkpointing
- Improve crawl frontier quality (canonicalization, deduping, domain quotas)
- Refine ranking (better snippets, phrase handling, spam filtering)

Detailed post-code-complete narrative in Appendix A.

APPENDIX

A Detailed Post Code-Complete Narrative (Scale + Quality)

We initially validated the full pipeline on a clean ∼10k-page corpus to ensure correctness across crawling, extraction, indexing, PageRank, and serving.

We then attempted a much larger crawl (hundreds of thousands of pages). At that scale, the crawler began to spend disproportionate time in **rabbit holes** (endless pagination, calendar pages, infinite parameterized URLs, near-duplicates), which harmed corpus quality and wasted crawl budget.

To address this, we restarted with:

- **Stricter frontier filtering** (URL canonicalization, depth/host constraints),
- **Blacklisting** of known trap patterns/domains,
- **More aggressive prioritization** so we keep diverse, high-signal pages.

The outcome was a clean ∼100k-page corpus that could be crawled in under one hour, which also made indexing + PageRank jobs more stable and reproducible.

B Architecture Details

B.1 System Architecture Diagram

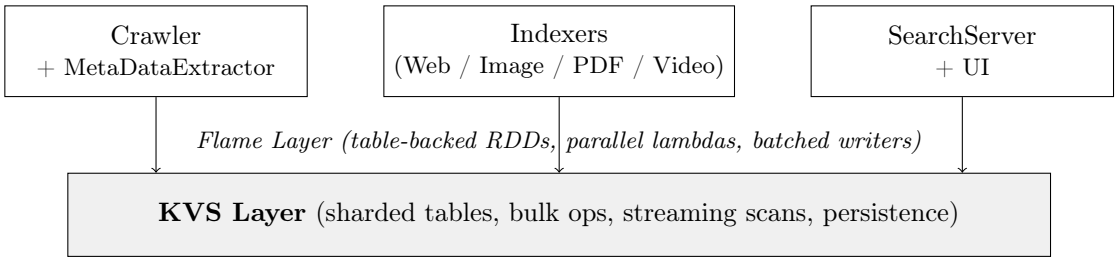

Figure 1: High-level architecture of Hashira Search.

B.2 Pipeline Overview

- **Crawler** downloads pages and stores raw HTML and crawl metadata in a persistent crawl table.
- **MetaDataExtractor** parses HTML to produce: (i) titles/descriptions, (ii) media link tables (image/PDF/video URLs with context), (iii) a filtered link graph for PageRank, (iv) inbound anchor text aggregated per destination.
- **Indexers** produce inverted indexes for web and media verticals.
- **PageRank** runs on the extracted link graph and stores a score per URL.
- **SearchServer** executes the online query flow: normalization, spell correction, candidate generation, ranking (lazy/incremental), caching, and rendering via the UI.

B.3 Key Data Structures (KVS Tables)

All data is stored as `Row` objects: a string key and a sparse set of named columns (values are `byte[]`). We use persistent tables with a `pt-` prefix so that long-running jobs can be restarted without losing state.

Table	Purpose
`pt-crawl`	Crawled pages (URL, HTML, crawl metadata)
`pt-meta-info`	URL hash → {URL, title, description}
`pt-outlinks`	Filtered outgoing link graph for PageRank (hashed URLs)
`pt-anchors`	Inbound anchor text per destination (`anchor:<src>` columns)
`pt-index`	Web inverted index: stem → aggregated postings with TF components + positions
`pt-docmagnitudes`	URL hash → document vector magnitude for cosine normalization
`pt-image-links` / `pt-pdf-links` / `pt-video-links`	Media URLs with aggregated context text (anchor/alt/caption/nearby)
`pt-image-index` / `pt-pdf-index` / `pt-video-index`	Media inverted indexes (anchor-context based; PDFs include positions)
`pt-pageranks`	URL hash → PageRank score

Table 1: Core persistent tables used in Hashira Search.

B.4 Implementation Details

- **Crawler Optimizations:** Streaming fetch (single GET with early abort), English detection (first 1700 bytes), bulk KVS ops with caching (robots.txt, crawl-status), frontier-based resumption
- **HTML Parsing & Token Processing:** JSoup extraction (title/meta/headers/body) → normalization → stopword removal → Snowball stemmer
- **Index Keying Strategy:** Hybrid `hash(prefix)_stem` format for balanced sharding while maintaining debuggability
- **Position Storage:** Web index stores exact term positions for proximity; PDF index uses approximate anchor-text positions
- **Batched Write Architecture:** Multi-level pooling (ParallelLambdaExecutor → BatchedKVSClient → BulkWritePool) with request merging and async I/O
- **Log-Based Writes:** Append-only logs for write-heavy workloads → batch merge in RAM → 10–100× compression for high-frequency terms

C Crawler

Our distributed web crawler is built on Flame and implements several key optimizations beyond the homework crawler for high throughput while maintaining politeness.

- **Streaming Fetch.** Instead of issuing separate `HEAD` and `GET` requests (two round-trips), we use a single `GET` request with early abort. Headers are inspected before downloading the body, and requests are aborted on timeouts, DNS failures, redirects, non-200 status codes, non-HTML content types, or non-English pages. This reduces network overhead by approximately 50%.
- **English Detection.** We read only the first 1,700 bytes of each page and check for the HTML `lang="en"` attribute. Pages without English tags are discarded immediately, keeping the index focused while reducing storage overhead.
- **Batched Writes.** Our `BatchedKVSClient` pools all `put()` calls across parallel lambda executions per Flame worker. Writes are buffered in memory and flushed in configurable batches, with duplicate row keys merged to eliminate redundancy. Asynchronous flushes allow crawler execution to continue while writes complete in the background.
- **Bulk KVS Operations with Caching.** Rather than issuing individual KVS reads per child URL, we perform bulk operations:
  - **Blacklist patterns:** Loaded once per JVM and cached in memory; all subsequent checks reuse the cached patterns.

- **Robots.txt:** A thread-safe in-memory cache is maintained. For uncached hosts, a single `bulkGetRows()` fetches all robots.txt entries, which are then cached.
- **Crawl-status checks:** A single `bulkGetRows()` to `pt-crawl` determines which child URLs have already been crawled.

This reduces KVS round-trips from $O(n)$ to $O(1)$ per processed page.

- **Frontier-Based Resumption.** The `pt-frontier-urls` table tracks discovered URLs using an `exploring` flag. On restart, the crawler resumes from unexplored frontier URLs, with 50% sampling applied to avoid overload. This enables crash recovery without losing crawl progress.

D Parallel Lambda Execution

D.1 Overview

The Parallel Lambda system implements a producer-consumer pattern inspired by Apache Spark/Flink, with multi-level batching and pooling to maximize throughput for distributed batch processing.

D.2 Architecture

D.3 Thread Pooling Configuration

Table 2: Thread pool configurations across system layers

Component	Location	Threads	Batch Size	Flush Timer
ParallelLambdaExecutor	Flame Worker	$n-1$ cores	1000 queue	–
BatchedKVSClient	Flame Worker	8 HTTP	1000 rows	6.5s
BulkWritePool	KVS Worker	16 disk I/O	500 rows	50ms

Key mechanisms:

- **Backpressure**: Bounded queue blocks producer when consumers lag
- **Request merging**: Multiple writes to same row → single merged row
- **Async I/O**: Non-blocking HTTP and disk writes via thread pools

D.4 Log-Based Writes for Eventual Consistency

For write-heavy workloads (e.g., indexing where word "the" appears in 100K+ docs), we use append-only logs instead of random writes: **Process**: Enable log mode → writes append to `table.log` → after job, call

Table 3: Log mode vs. normal mode

Aspect	Normal	Log Mode
Write pattern	Random files	Sequential append
Read-before-write	Yes	No
Disk seeks	Many	One

`convertLogToTable()` which reads log in 100MB chunks, merges by key in RAM, writes once per unique key. Achieves 10–100× compression.

D.5 Eventual Consistency Trade-offs

We accept eventual consistency (reads may be stale $\leq$ 6.5s) for 10–50× throughput gains. This works because:

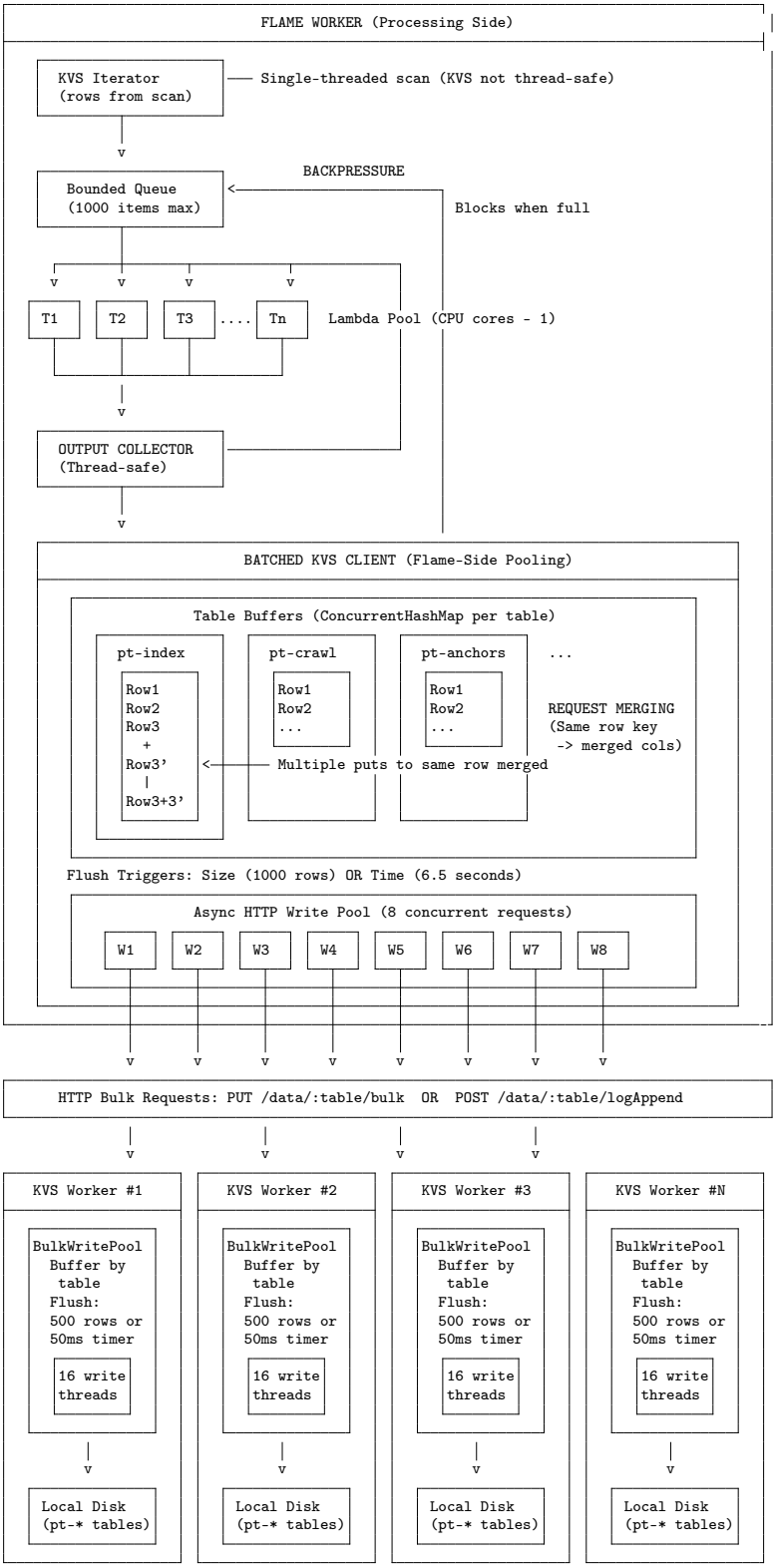

Figure 2: Batched write architecture: Flame Worker → BatchedKVSClient → KVS Workers with multi-level pooling.

- **Indexing/PageRank**: No reads during writes
- **Guarantees**: Row atomicity, column merging, flush-on-close, no data loss

D.6 Available Parallel Operations

Table 4: Parallel RDD operations

RDD	Operations
FlameRDD	`fromTableParallel`, `flatMapParallel`, `flatMapToPairParallel`, `mapToPairParallel`, `filterPar`
FlamePairRDD	`flatMapParallel`, `flatMapToPairParallel`, `foldByKeyParallel`, `cogroupParallel`, `joinParalle`

D.7 Performance

Table 5: Throughput improvements

Scenario	Sequential	Parallel	Speedup
Indexer	50 docs/sec	2,000 docs/sec	40×
PageRank iteration	30 min	3 min	10×

E Detailed Ranking Specifications

E.1 Per-Feature Scoring

TF–IDF. For a query Q and document d:

$$\mathrm{idf}(w) = \log_{10}\left(\frac{N}{\max(\mathrm{df}(w), 1)}\right), \quad \mathrm{tf}(w, d) \text{ from stored normalized term frequencies,}$$

$$(Q, d) = \frac{\langle \vec{q}, \vec{d} \rangle}{\|\vec{q}\| \cdot \|\vec{d}\|},$$

where $\|\vec{d}\|$ is read from `pt-docmagnitudes`.
Field score. We compute a weighted combination of field-specific TFs (title/meta/header/body) and normalize to $[0, 1]$.
Proximity. Using positions stored per term, we compute a penalty/bonus based on the minimum span covering all query terms (tighter span $\Rightarrow$ higher score). The result is normalized to $[0, 1]$.
URL + anchor. We tokenize URL paths and use inbound anchor text accumulated in `pt-anchors`; both are scored similar to TF signals, then normalized.
PageRank. We store a PageRank value per URL and normalize it across candidates to $[0, 1]$ for merging.

E.2 Final Ranking Formulas

Web results:

$$\text{finalScore} = 0.45 \cdot \text{cosine} + 0.15 \cdot \text{field} + 0.10 \cdot \text{proximity} + 0.10 \cdot \text{urlAnchor} + 0.20 \cdot \text{pageRankNorm}.$$

Images/Videos:

$$\text{finalScore} = 0.75 \cdot \text{cosine} + 0.25 \cdot \text{proximity}.$$

PDFs:

$$\text{finalScore} = 0.70 \cdot \text{cosine} + 0.30 \cdot \text{proximity}.$$

F Performance Evaluation Details

F.1 Speed Metrics

Stage	Input size	Runtime	Throughput
Crawler	100,000 pages	< 30 mins	> 50 pages/sec
MetaDataExtractor	100,000 pages	*< 30 mins*	*N/A : separate reads and writes*
Indexer.a (web)	100,000 pages	*< 30 mins*	*> 50 pages/sec*
Indexer.b (PDFs, Images, Videos)	*20K items combined*	*Sub 10 mins*	*N/A*
Pagerank	*100,000 URLs*	*20 mins*	*iters: 7-10*

Table 6: End-to-end performance summary.

F.2 Experiment Methodology

Experiment	What we varied	What we measured
Crawl scaling	#crawler threads, #nodes	pages/sec, error rate, domain skew
Index scaling	#Flame workers/threads	docs/sec, output table size
PageRank scaling	$N \in \{500, 2500, 5000, \dots\}$; #nodes	sec/iter, iters to converge, peak memory
Query latency	expensive features on/off	p50/p95 latency, cache hit rate

Table 7: Experiment methodology

F.3 PageRank Experiments

Algorithm. We run iterative PageRank with decay factor 0.85 and a teleportation term 0.15.
Key optimization (join elimination). Naive join/cogroup-style implementation materializes large intermediate structures and increases memory pressure, so we eliminated it by doing direct (batched) KVS lookup of aggregated transfer values during the update step, reducing peak memory and improving iteration time.
Convergence check optimization. Instead of collecting all ranks to the coordinator ($O(N)$), we aggregate convergence stats on workers and collect only constant-sized summary each iteration ($O(1)$ work).

F.4 Query Latency (Online Serving)

Latency depends on:

- number of candidate documents pulled from `pt-index`,
- whether proximity is enabled (positions parsing),
- whether debug mode is enabled (more score components computed/returned),
- cache hit rate for the same corrected query.

Infinite scroll stays responsive because we score incrementally, cache, and only compute deeper results when requested.

G Indexing, Ranking, and PageRank Optimizations

This appendix summarizes the key optimizations that made the **indexing**, **ranking**, and **PageRank** stages fast enough to run at scale.

G.1 Indexing Optimizations

Stopword removal + stemming (smaller index, faster queries). During indexing we:

- remove stopwords early (reduces postings volume and noise),
- apply stemming and store **only stemmed tokens** in the inverted index,
- store document magnitudes based on the same stemmed vocabulary for consistent cosine normalization.

This reduces table size (fewer unique keys) and lowers query-time candidate expansion, since search is also performed on stemmed tokens.

Linear-time proximity/phrase processing via two pointers (avoid $O(n^2)$). For proximity scoring and multi-term matching, naive merging of position lists can become quadratic when lists are long. Instead, we use a **two-pointer merge** over sorted position arrays (and generalized multi-list merges) so that:

- computing minimum span / nearest co-occurrence is **near-linear** in total positions,
- we avoid nested scanning of position lists ($O(n^2)$) on frequent terms.

Parallel indexing with table-backed operations. Index construction is implemented using Flame parallel operators (e.g., parallel table scans and parallel map/flatMap pipelines), so CPU-heavy parsing (tokenization, stopword filtering, stemming) runs concurrently across workers/threads. Index output writes are batched to minimize HTTP calls and disk flush overhead.

G.2 Ranking Optimizations (Query-Time)

Search operates on stemmed vocabulary. At query time, we normalize the query in the same way as indexing (stopword removal + stemming), then fetch postings only for those stems. This keeps query processing aligned with the stored index and avoids wasteful lookups.

Fast proximity using stored positions + linear merges. Because postings include term positions, proximity scoring does not require scanning full documents. We compute proximity using the same two-pointer/merge strategy over position lists, which is efficient even for long documents.

Incremental scoring for responsiveness. To maintain low latency for the first page of results, we:

- score an initial candidate slice synchronously,
- continue scoring deeper candidates lazily (supporting infinite scroll),
- cache intermediate results keyed by the corrected query so repeated requests reuse work.

G.3 PageRank Optimizations (Iterative Job)

Avoiding cogroup. A cogroup/join-like approach tends to load large state and transfer structures, increasing memory usage and slowing each iteration. We instead:

- compute aggregated incoming transfer per URL as a table,
- update ranks by direct (batched) KVS lookup of that transfer for each page,
- keep the state table compact and streaming-friendly.

Convergence detection. We aggregate convergence statistics on workers and send only a constant-sized summary to the coordinator each iteration, avoiding O(N) collects.

Parallel execution and batched I/O. PageRank iterations use parallel table scans and batched reads/writes so each iteration is dominated by sequential streaming rather than per-row RPC overhead.

H AWS Deployment

H.1 Infrastructure as Code (Terraform)

All cloud infrastructure is provisioned via **Terraform**, enabling reproducible, version-controlled deployments on AWS.

H.1.1 Directory Structure

```
terraform/
├── main.tf              # EC2 instances, security groups
├── variables.tf         # Configurable parameters
├── outputs.tf           # Exported IPs for scripts
├── terraform.tfvars     # Production values (gitignored)
└── scripts/
    ├── run-kvs-aws.sh
    ├── run-flame-aws.sh
    ├── run-indexer-aws.sh
    ├── run-indexer-pdf-aws.sh
    ├── run-indexer-video-aws.sh
    └── run-indexer-with-images-aws.sh
```

H.2 Final Production Configuration

H.2.1 Co-Located Architecture

KVS and Flame services are **co-located** on the same EC2 instances to minimize network overhead and simplify deployment:

- **Coordinator instance**: Runs both KVS Coordinator and Flame Coordinator.
- **Worker instances**: Each runs both a KVS Worker and a Flame Worker process.

H.2.2 Instance Specifications

Role	Instance Type	Count	vCPU / RAM	Services
Coordinator	`t3.small`	1	2 vCPU / 2 GB	KVS Coord (:8000) + Flame Coord (:9000)
Worker	`t3.large`	9	2 vCPU / 8 GB	KVS Worker (:8001) + Flame Worker (:9001)

Total: 10 EC2 instances (1 coordinator + 9 workers).

H.2.3 AMI & Region

```
ami_id     = "ami-0c02fb55956c7d316"  # Amazon Linux 2 (us-east-1)
region     = "us-east-1"
key_pair   = "cis5550-fa25"
```

H.2.4 Terraform Variable Definitions

```
# variables.tf
variable "worker_count"        { default = 9 }
variable "coordinator_type"    { default = "t3.small" }
variable "worker_type"         { default = "t3.large" }
variable "ami_id"              { default = "ami-0c02fb55956c7d316" }
variable "key_pair_name"       { default = "cis5550-fa25" }
```

H.3 Security Groups

Name	Inbound	Source	Purpose
`sg-cluster`	TCP 8000–8010, 9000–9010	VPC CIDR	KVS + Flame internal traffic
`sg-http`	TCP 80, 443	0.0.0.0/0	Frontend / API access
`sg-ssh`	TCP 22	Your IP	Deployment script access

All outbound traffic is allowed (egress: 0.0.0.0/0).

H.4 Terraform Outputs

Scripts consume these outputs via `terraform output -json`:

```
# outputs.tf
output "coordinator_ip" {
  value = aws_instance.coordinator.public_ip
}
output "worker_ips" {
  value = aws_instance.worker[*].public_ip
}
```

Since KVS and Flame are co-located, both coordinators share the same IP, and each worker IP hosts both services.

H.5 Deployment Scripts

Each script follows the same pattern: fetch IPs from Terraform, build JARs, deploy via SSH.

H.5.1 Script Overview

Script	Purpose
`run-cluster-aws.sh`	Starts both KVS + Flame (coordinator on :8000/:9000, workers on :8001/:9001)
`run-indexer-aws.sh`	Submits `IndexerJsoup` job for HTML pages
`run-indexer-pdf-aws.sh`	Submits `IndexerPDF` job for PDF documents
`run-indexer-video-aws.sh`	Submits `IndexerVideo` job for video metadata
`run-indexer-with-images-aws.sh`	Submits `IndexerWithImages` job

H.5.2 Cluster Startup Script

Since KVS and Flame are co-located, a single script deploys both:

```
#!/bin/bash
set -e
cd "$(dirname "$0")/.."

# Fetch IPs from Terraform
COORD=$(terraform -chdir=terraform output -raw coordinator_ip)
WORKERS=$(terraform -chdir=terraform output -json worker_ips | jq -r '.[]')

# Build JARs
mvn -q package -DskipTests
```

```
# Start coordinator (both KVS + Flame)
scp -i "$KEY" target/*.jar ec2-user@$COORD:~/
ssh -i "$KEY" ec2-user@$COORD "
  nohup java -jar kvs-coordinator.jar 8000 > kvs-coordinator.log 2>&1 &
  nohup java -jar flame-coordinator.jar 9000 $COORD:8000 > flame-coordinator.log 2>&1 &
"

# Start workers (both KVS + Flame on each instance)
WORKER_ID=1
for IP in $WORKERS; do
  scp -i "$KEY" target/*.jar ec2-user@$IP:~/
  ssh -i "$KEY" ec2-user@$IP "
    nohup java -jar kvs-worker.jar $COORD:8000 8001 worker_$WORKER_ID > kvs-worker.log 2>&1 &
    nohup java -jar flame-worker.jar $COORD:9000 9001 > flame-worker.log 2>&1 &
  "
  ((WORKER_ID++))
done
```

H.5.3 Indexer Job Submission

```
#!/bin/bash
# run-indexer-aws.sh
COORD=$(terraform -chdir=terraform output -raw coordinator_ip)
java -cp indexer.jar cis5550.jobs.IndexerJsoup "$COORD:9000" "$COORD:8000"
```

H.6 Provisioning Lifecycle

1. `terraform init` — Initialize providers and S3 backend.
2. `terraform plan` — Preview infrastructure changes.
3. `terraform apply -auto-approve` — Provision 10 EC2 instances.
4. `./scripts/run-cluster-aws.sh` — Deploy and start KVS + Flame (co-located).
5. `./scripts/run-indexer-aws.sh` — Submit indexing jobs.
6. `terraform destroy` — Tear down all resources (cost control).

H.7 Monitoring

H.7.1 Queue Depth Monitoring

`monitor-queues-aws.sh` tails logs from all instances in real time:

```
#!/bin/bash
# Fetch all worker IPs (each runs both KVS + Flame)
WORKERS=$(terraform output -json worker_ips | jq -r '.[]')

# Tail both logs from each worker instance
for IP in $WORKERS; do
  # KVS worker log (blue)
  ssh -i "$KEY" ec2-user@$IP "tail -f ~/kvs-worker.log" | \
    sed "s/^/\x1b[34m[KVS $IP]\x1b[0m /" &
  # Flame worker log (green)
  ssh -i "$KEY" ec2-user@$IP "tail -f ~/flame-worker.log" | \
    sed "s/^/\x1b[32m[FLAME $IP]\x1b[0m /" &
done
wait
```

H.7.2 Log Files

Service	Log Path
KVS Coordinator	`~/kvs-coordinator.log`
KVS Worker	`~/kvs-worker.log`
Flame Coordinator	`~/flame-coordinator.log`
Flame Worker	`~/flame-worker.log`

Color coding: KVS (blue), Flame (green), warnings (yellow), errors (red).

H.8 Cost Optimization

- **Co-location**: Running KVS + Flame on same instances reduces total instance count.
- **Spot instances**: Workers use spot pricing (~70% savings).
- **Auto-shutdown**: `terraform destroy` after indexing completes.
- **Right-sizing**: Coordinator uses `t3.small`, workers use `t3.large`.

Estimated cost: ~$0.85/hour for full cluster (10 instances).

- $1 \times$ `t3.small` @ $0.0208/hr = $0.02
- $9 \times$ `t3.large` @ $0.0832/hr = $0.75

I Use of AI Coding Assistants

We used AI tools as a productivity aid for debugging and documentation drafting. All suggestions were reviewed and tested; final design decisions and code changes were made by the team.